

ROADMAP DEL PROYECTO

Sistema de Gestión de Ventas

DB_GestionVentas

Resumen retrospectivo del desarrollo realizado durante el curso

Enfoque del documento

Este documento presenta el proyecto como una ruta de trabajo ya ejecutada: desde el análisis inicial y el diseño relacional hasta la automatización, las pruebas, el respaldo y la preparación para portafolio.

Autor: Jostyn Mendoza

Tecnologías: SQL Server · SQL Server Management Studio · T-SQL

1. Contexto y propósito del proyecto

Durante el curso se desarrolló una base de datos relacional orientada a representar el proceso de ventas de una empresa con varias sucursales. El proyecto se planteó como un caso práctico para integrar modelado, consultas, automatización y control de operaciones dentro de SQL Server.

1.1 Problema abordado

El sistema debía evitar que la información comercial quedara aislada. Para ello fue necesario relacionar clientes, empleados, pedidos, productos, pagos e inventario, manteniendo reglas de integridad y un control automático del stock.

1.2 Objetivo alcanzado

Se implementó una base de datos capaz de registrar operaciones de venta, consultar información de negocio, validar restricciones, automatizar tareas y proteger procesos mediante transacciones.

1.3 Resultado general

- 10 tablas principales relacionadas mediante claves primarias y foráneas.
- Consultas básicas, intermedias y analíticas con JOIN, subconsultas, CTE y funciones de ventana.
- Vistas, funciones y procedimientos almacenados reutilizables.
- Triggers para actualización y descuento automático de stock.
- Transacciones con COMMIT, ROLLBACK, TRY/CATCH y THROW.
- Carga masiva de datos, pruebas funcionales y backup de la base.

Idea central

El proyecto evolucionó desde una base estructural simple hasta un sistema que aplica reglas de negocio y automatizaciones dentro de la propia base de datos.

2. Roadmap de desarrollo realizado

El trabajo se ejecutó por etapas. Cada fase agregó una capacidad nueva al sistema y se apoyó en lo construido anteriormente.

1	Análisis y alcance Se definieron las entidades necesarias y el flujo principal de ventas. Entregables: Modelo conceptual inicial y alcance funcional.
2	Diseño relacional Se identificaron PK, FK y relaciones entre clientes, pedidos, productos, empleados, sucursales, pagos e inventario. Entregables: Diagrama ER y estructura de tablas.
3	Implementación de tablas Se crearon tablas con IDENTITY, NOT NULL, UNIQUE, CHECK, DEFAULT y FOREIGN KEY. Entregables: Scripts de creación e integridad.
4	Carga inicial y consultas Se insertaron datos y se desarrollaron SELECT, WHERE, ORDER BY, JOIN, GROUP BY, HAVING y subconsultas. Entregables: Datos de prueba y consultas operativas.

2.1 Roadmap - continuación

En la segunda mitad del desarrollo se incorporaron recursos de análisis, reutilización de lógica, automatización y control transaccional.

5	Análisis intermedio Se incorporaron CTE y Window Functions como ROW_NUMBER, RANK, LAG y SUM() OVER(). Entregables: Consultas de análisis y ranking.
6	Objetos reutilizables Se construyeron vistas, funciones y procedimientos almacenados. Entregables: Capa de consultas y operaciones reutilizables.
7	Automatización y control Se implementaron triggers, validación de stock, transacciones y manejo de errores. Entregables: Automatización y consistencia transaccional.
8	Pruebas, volumen y cierre Se cargaron más datos, se verificó el comportamiento del sistema, se generó backup y documentación. Entregables: Base final, diagrama, PDF y repositorio.

3. Diseño de la base de datos

El modelo final quedó organizado alrededor del flujo Cliente → Pedido → Detalle_Pedido → Producto. A partir de ese núcleo se conectaron las tablas de empleados, sucursales, pagos, categorías, proveedores e inventario.

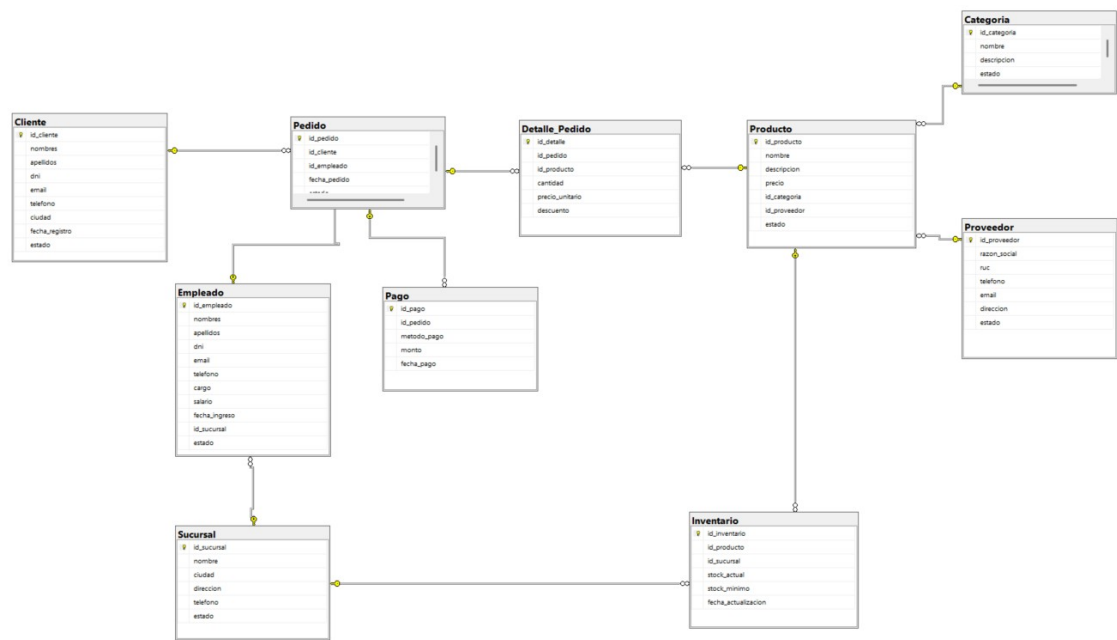


Figura 1. Diagrama final de relaciones utilizado en la presentación del proyecto.

3.1 Responsabilidad de las tablas

Grupo	Tablas	Función
Ventas	Cliente, Pedido, Detalle_Pedido, Pago	Registrar quién compra, qué compra y cómo paga.
Productos	Producto, Categoria, Proveedor	Mantener el catálogo y su procedencia.
Operación	Empleado, Sucursal	Identificar quién atiende y desde qué sede.
Control	Inventario	Gestionar stock por producto y sucursal.

4. Evolución técnica del proyecto

4.1 Consultas y análisis

Después de crear la estructura se desarrollaron consultas orientadas a responder preguntas reales del sistema, no solo a mostrar registros.

- Clientes con mayor cantidad de pedidos.
- Productos y categorías más vendidos.
- Ventas por empleado y por sucursal.
- Totales por método de pago.
- Productos con stock bajo.
- Ranking de productos según ingresos.

```
SELECT
  p.nombre,
  p.precio,
  RANK() OVER (ORDER BY p.precio DESC) AS ranking
FROM Producto p;
```

4.2 Vistas, funciones y procedimientos

Objeto	Ejemplos implementados
Vistas	vw_EmpleadosSucursal, vw_ProductosDetalle, vw_DetalleVentas, vw_ProductosStockBajo
Funciones	fn_CalcularTotalDescuento, fn_TotalPedido, fn_StockTotalProducto
Procedimientos	sp_RegistrarCliente, sp_ConsultarPedidosCliente, sp_ActualizarEstadoPedido

Con estos objetos se logró reutilizar lógica frecuente y separar tareas de consulta, cálculo y modificación.

5. Automatización y control de operaciones

5.1 Triggers implementados

La etapa más automatizada del proyecto se alcanzó con triggers. El primero actualiza la fecha del inventario cuando cambia el stock. El segundo valida y descuenta el stock al registrar una venta.

Flujo del trigger de venta

INSERT en Detalle_Pedido → identificar Pedido → identificar Empleado y Sucursal → buscar Inventario → validar stock → descontar cantidad. Si no existe stock suficiente, se lanza un error.

```
IF EXISTS (...)  
BEGIN  
    THROW 50001,  
        'Stock insuficiente para realizar la venta.',  
        1;  
END;
```

5.2 Transacciones

Para evitar ventas incompletas se trabajó con transacciones. El bloque TRY intenta registrar pedido, detalle y pago; si todo funciona se ejecuta COMMIT. Si ocurre un error, CATCH ejecuta ROLLBACK y revierte los cambios.

```
BEGIN TRY  
    BEGIN TRANSACTION;  
    -- crear pedido  
    -- registrar detalle  
    -- registrar pago  
    COMMIT;  
END TRY  
BEGIN CATCH  
    IF @@TRANCOUNT > 0  
        ROLLBACK;  
    THROW;  
END CATCH;
```

Resultado

La base no queda en un estado intermedio: una venta se confirma completa o se revierte completa.

6. Validación, cierre y presentación

6.1 Carga masiva y pruebas

Para comprobar el comportamiento del sistema con mayor volumen, se incorporó un script adicional que agregó clientes, sucursales, categorías, proveedores, empleados, productos, inventario, pedidos, detalles y pagos. Esta carga permitió probar consultas y automatizaciones con un escenario más cercano a uno real.

Prueba realizada	Comportamiento esperado
Venta con stock	Se registra el detalle y el stock disminuye.
Venta sin stock	THROW genera error y la transacción se revierte.
COMMIT	Los cambios quedan confirmados.
ROLLBACK	Los cambios de la transacción se deshacen.
Consultas analíticas	Devuelven rankings, totales y agrupaciones con mayor variedad de datos.

6.2 Backup

Como parte del cierre se creó un script de BACKUP DATABASE para conservar una copia completa del estado de la base y permitir su restauración o uso en pruebas posteriores.

```
BACKUP DATABASE DB_GestionVentas
TO DISK = 'C:\Backups\DB_GestionVentas.bak'
WITH INIT;
```

6.3 Conclusión de la presentación

El proyecto permitió recorrer de forma progresiva los principales componentes de SQL Server: diseño relacional, consultas, análisis, programación en T-SQL, automatización y control transaccional. El resultado final fue una base de datos funcional, documentada y preparada para publicarse como proyecto de portafolio.

Competencias demostradas

Modelado relacional · SQL intermedio · JOIN · subconsultas · CTE · Window Functions · vistas · funciones · procedimientos · triggers · transacciones · integridad de datos · backup.